

TEMA N° 1



TEMA 1 INTRODUCCIÓN A LAS TECNOLOGÍAS MÓVILES



EN ESTE TEMA SERÁS CAPAZ DE...

1. **Comprender a profundidad la evolución del hardware móvil y sus restricciones críticas**, identificando cómo las limitaciones de memoria, procesamiento y energía impactan directamente en el rendimiento de los sistemas de software en entornos reales.
2. **Evaluar y seleccionar estratégicamente la arquitectura de desarrollo óptima (Nativa, Híbrida o PWA)**, basándote en las necesidades técnicas de los usuarios, las características del mercado local y las condiciones de conectividad de tu entorno.
3. **Configurar de manera profesional entornos de desarrollo técnico avanzados**, dominando la instalación de SDKs, la automatización mediante consolas de comandos y el despliegue controlado tanto en emuladores de alta fidelidad como en dispositivos físicos reales.



PRÁCTICA



PREGUNTA DETONANTE

¿Te has puesto a pensar qué sucede detrás de la pantalla de tu teléfono cuando intentas abrir una aplicación pesada mientras caminas por una zona con baja cobertura telefónica, y por qué algunos teléfonos se calientan o cierran las aplicaciones de forma inesperada?

EL DESAFÍO TECNOLÓGICO EN EL MUNICIPIO DE PAILÓN

Imagina que la **Asociación de Productores y Comerciantes de la Chiquitanía**, cuya base operativa se encuentra en el **Barrio Central Fátima** (frente a la plaza principal e iglesia de Pailón), ha decidido modernizar la distribución de sus productos agrícolas y lácteos. Actualmente, coordinan de manera manual los despachos hacia los centros de distribución ubicados en el **Barrio 24 de Septiembre** (cerca de la estación de tren y los campos deportivos) y las entregas directas a las familias en los barrios de expansión rápida como **Jardines de Pailón, Oto Waver, San Expedito y Barrio Saó**.

Para solucionar este caos logístico, la asociación ha acudido a los estudiantes del cuarto semestre del **CEA Herman Ortiz Camargo**, aprovechando el nuevo módulo educativo equipado en el Barrio Saó. El objetivo es diseñar un ecosistema móvil que permita a los transportistas recibir rutas de entrega, registrar pagos y actualizar el inventario en tiempo real.

Sin embargo, como Técnico encargado del proyecto, te enfrentas a una serie de restricciones del mundo real que dificultan el desarrollo:

1. **Heterogeneidad de Dispositivos:** Los transportistas y comerciantes utilizan teléfonos celulares propios que van desde dispositivos de gama baja (con más de 5 años de antigüedad, procesadores antiguos y apenas 2 GB de memoria RAM) hasta teléfonos de gama media adquiridos recientemente en las ferias comerciales de Pailón.
2. **Conectividad Intermittente:** Mientras que en el Barrio Central Fátima y el Barrio 27 de Mayo (cerca del coliseo municipal) la señal de datos móviles es estable, en las rutas rurales que conectan con **Las Misiones, San José**, o en el límite de los barrios **María Belén** y



Ovidio Barberi, la conexión a internet cae por completo o es sumamente lenta (redes Edge/2G).

3. **Consumo de Recursos en Climas Cálidos:** En Pailón, las temperaturas superan fácilmente los 38°C durante gran parte del año. Los teléfonos instalados en los tableros de las camionetas y motocicletas sufren de sobrecalentamiento constante, lo que provoca que el sistema operativo reduzca drásticamente la velocidad del procesador para enfriarse, cerrando aplicaciones en segundo plano para ahorrar energía.

La asociación necesita que Determine de forma justificada qué tecnología de desarrollo se debe emplear, cómo se estructurará la aplicación para que no consuma los escasos megabytes de internet de los usuarios, y cómo asegurar que la aplicación funcione de manera fluida sin bloquear los teléfonos más antiguos del personal de distribución.



TEORÍA



1.1. EVOLUCIÓN, CARACTERÍSTICAS Y LIMITACIONES DE HARDWARE MÓVIL

El desarrollo de software para dispositivos móviles requiere un cambio de paradigma radical en comparación con el desarrollo para computadoras de escritorio. En las computadoras de escritorio, los recursos de energía y procesamiento son relativamente abundantes; en los dispositivos móviles, el hardware está severamente condicionado por el espacio físico, la disipación térmica y la duración de la batería.

La evolución del SoC (System on Chip)

A diferencia de las computadoras tradicionales, donde la CPU, la tarjeta gráfica (GPU) y la memoria RAM se encuentran en componentes separados dentro de una tarjeta madre, los dispositivos móviles agrupan todos estos componentes en un solo circuito integrado conocido como **SoC (System on Chip)**. Esta arquitectura ultra compacta reduce las distancias físicas entre componentes, disminuyendo drásticamente el consumo de energía y la generación de calor.



Los procesadores móviles utilizan predominantemente la arquitectura **ARM (Advanced RISC Machine)** en lugar de la arquitectura x86 (común en computadoras Intel/AMD). La arquitectura ARM se basa en un conjunto de instrucciones reducidas (RISC), lo que significa que ejecuta instrucciones más simples y optimizadas, consumiendo una fracción de la energía eléctrica. Para gestionar el rendimiento de forma eficiente, los procesadores modernos utilizan la arquitectura **big.LITTLE** de ARM, la cual combina núcleos de alta eficiencia (LITTLE) para tareas secundarias o en reposo (como recibir mensajes de texto o sincronizar la hora en segundo plano) con núcleos de alto rendimiento (big) que se activan únicamente cuando el usuario ejecuta tareas pesadas (como juegos o procesamiento de mapas).

Limitaciones críticas de hardware para el Técnico en Sistemas

1. **Memoria RAM Restringida y Gestión de Procesos:** Aunque un teléfono de gama media actual posea 4 GB u 8 GB de RAM, el sistema operativo (especialmente Android) reserva un porcentaje sustancial para sus servicios internos. El software desarrollado debe optimizar el uso de objetos en memoria. Cuando el sistema operativo detecta que la memoria RAM está llegando a su límite, activa un mecanismo agresivo llamado *Low Memory Killer (LMK)*, el cual destruye procesos en segundo plano sin previo aviso para garantizar la estabilidad de la interfaz del usuario.



2. **Almacenamiento Flash y Ciclos de Lectura/Escritura:** Los dispositivos móviles utilizan almacenamiento de tipo Flash (eMMC o UFS). Aunque son rápidos para la lectura, escribir constantemente datos en bases de datos locales sin optimización degrada la vida útil del almacenamiento y ralentiza la velocidad general del teléfono.
3. **Restricciones Térmicas y Degradación de Batería:** Los teléfonos móviles no tienen ventiladores. Toda la disipación de calor se realiza de forma pasiva a través del chasis del dispositivo. En zonas de altas temperaturas como Pailón, si una aplicación móvil mal optimizada satura continuamente la CPU, el dispositivo entrará en un estado de *Thermal Throttling* (estrangulamiento térmico), reduciendo la frecuencia del procesador hasta en un 50% para evitar daños estructurales, lo que destruye la fluidez de la aplicación de cara al usuario.

1.2. COMPARATIVA DE SISTEMAS OPERATIVOS (ANDROID VS. IOS)

El mercado global de software móvil está polarizado por dos sistemas operativos dominantes. Para un Técnico en Sistemas Informáticos, entender las diferencias arquitectónicas e ideológicas entre ambos es fundamental antes de iniciar cualquier línea de producción de software.

Característica	Android (Google)	iOS (Apple)
Núcleo (Kernel)	Basado en un Kernel de Linux modificado.	Basado en el Kernel Darwin (XNU), derivado de UNIX/BSD.
Ecosistema	Código abierto (AOSP). Licencia libre para fabricantes.	Código cerrado y propietario. Exclusivo de hardware Apple.
Lenguajes Oficiales	Kotlin y Java.	Swift y Objective-C.
Fragmentación	Muy Alta. Miles de modelos, pantallas y versiones activas simultáneamente.	Muy Baja. Pocos modelos controlados directamente por el fabricante.
Modelo de Distribución	Tiendas oficiales (Google Play) y repositorios alternativos (.APK).	Únicamente a través de la App Store oficial de Apple.
Gestión de Memoria	Recolector de Basura (Garbage Collector) en tiempo de ejecución.	Conteo Automático de Referencias (ARC) en tiempo de compilación.



Análisis de Impacto para el Contexto Técnico

En nuestra realidad regional dentro de la provincia Chiquitos, el dominio de Android es casi absoluto debido a la accesibilidad económica del hardware. No obstante, el desarrollo para Android presenta el desafío crítico de la **fragmentación**. Un Técnico debe diseñar interfaces de usuario responsivas que se adapten a resoluciones de pantalla de diversas densidades y asegurar que el código sea compatible de forma retroactiva con versiones anteriores de las APIs de Android (idealmente desde Android 7.0 Nougat en adelante), garantizando la inclusión de usuarios con hardware desactualizado.

Por otro lado, la arquitectura de iOS gestiona la memoria a través de ARC, lo que significa que el compilador determina exactamente cuándo liberar la memoria antes de que la aplicación se ejecute, resultando en un rendimiento sumamente fluido incluso con especificaciones técnicas nominalmente más bajas de memoria RAM. Sin embargo, los costos de hardware, licencias de desarrollo y restricciones de distribución hacen que el desarrollo nativo para iOS sea un mercado de nicho en proyectos socioproductivos locales.

1.3. DIFERENCIAS ENTRE DESARROLLO NATIVO, HÍBRIDO Y PWA

Cuando abordamos la construcción del software solicitado por la Asociación de Productores de Pailón, la decisión arquitectónica definirá el éxito o fracaso del proyecto. Existen tres enfoques principales para el desarrollo de software móvil.

1. Desarrollo Nativo

Consiste en escribir la aplicación utilizando los lenguajes, herramientas y SDKs específicos provistos por los creadores de cada sistema operativo: Kotlin o Java dentro de Android Studio para Android; Swift dentro de Xcode para iOS.

1. **Ventajas:** Rendimiento máximo, acceso sin restricciones a todas las APIs del hardware (cámara, sensores biométricos, Bluetooth, GPS de alta precisión, almacenamiento seguro), y control absoluto del ciclo de vida de la aplicación.



2. **Desventajas:** Alto costo y tiempo de desarrollo. Si se requiere la aplicación para ambas plataformas, es necesario programar y mantener dos códigos fuente completamente distintos.

2. Desarrollo Híbrido / Multiplataforma (Cross-Platform)

Permite escribir un único código fuente que luego se compila o interpreta para ejecutarse en múltiples sistemas operativos (Android e iOS). Frameworks modernos como **Flutter** (usando el lenguaje Dart) o **React Native** (usando JavaScript/TypeScript) lideran esta categoría.

1. **Ventajas:** Código único reutilizable hasta en un 90%, reducción drástica de tiempos de desarrollo, mantenimiento unificado. En el caso de Flutter, el rendimiento es muy cercano al nativo debido a su motor de renderizado propio (Impeller/Skia).
2. **Desventajas:** El acceso a características de hardware muy específicas o nuevas APIs del sistema operativo depende de que la comunidad o la empresa desarrolladora actualice los plugins intermedios ("puentes nativos"), lo que puede generar retardos o overhead en dispositivos de gama baja.

3. Aplicaciones Web Progresivas (PWA - Progressive Web Apps)

Son aplicaciones web estándar construidas con tecnologías tradicionales (HTML5, CSS3, JavaScript) que emplean tecnologías del navegador web llamadas **Service Workers**. Estos componentes actúan como proxies de red locales, permitiendo que la aplicación se instale en el teléfono directamente desde el navegador, funcione completamente sin conexión a internet, almacene datos de manera local e incluso reciba notificaciones push.



1. **Ventajas:** Peso ultra ligero (generalmente menos de 2 MB), no requiere descarga desde tiendas oficiales, actualización instantánea en el servidor sin intervención del usuario, compatibilidad universal con cualquier dispositivo que tenga un navegador moderno.
2. **Desventajas:** El acceso al hardware está severamente limitado por las políticas de seguridad del navegador. No tiene acceso a APIs de bajo nivel, Bluetooth interno, o control avanzado del procesador. El rendimiento gráfico está restringido por el motor de renderizado de la vista web del dispositivo.

Criterio	Desarrollo Nativo	Desarrollo Híbrido (Flutter/React Native)	PWA (Aplicación Web Progresiva)
Rendimiento	Excelente / Máximo.	Alto / Muy Bueno.	Moderado / Dependiente del Navegador.
Costo de Desarrollo	Elevado (Doble Equipo).	Eficiente (Un solo equipo).	Muy Económico e Inmediato.
Acceso al Hardware	Total y Directo.	Alto (Mediante Plugins).	Limitado (APIs Web básicas).



Peso del Archivo	Moderado.	Pesado (Incluye motores base).	Ultra Ligero (KiloBytes / MegaBytes mínimos).
Funcionamiento Offline	Diseñado nativamente por código.	Diseñado nativamente por código.	Excelente mediante Service Workers.

1.4. INSTALACIÓN DE SDKS Y ENTORNOS DE DESARROLLO (EJ. ANDROID STUDIO)

Para iniciar la producción de software en los laboratorios del CEA Herman Ortiz Camargo, es indispensable preparar la estación de trabajo bajo estándares profesionales. El entorno de desarrollo integrado (IDE) estándar de la industria es **Android Studio**, el cual se ejecuta sobre las herramientas del **Android SDK (Software Development Kit)**.

Requisitos del Sistema e Instalación de Dependencias Base

En sistemas operativos basados en Linux o Windows, antes de instalar Android Studio, es sumamente recomendable garantizar la presencia del Entorno de Ejecución de Java y herramientas de compilación esenciales.

A continuación, se detalla el script de comandos técnicos para la terminal que automatiza la verificación y preparación de variables de entorno globales dentro de la estación de trabajo:

Bash

```
# Actualizamos los repositorios del sistema operativo local para asegurar
descargas seguras sudo apt update && sudo apt upgrade -y
# Instalamos el Kit de Desarrollo de Java (OpenJDK 17), versión LTS recomendada
para desarrollo móvil estable sudo apt install openjdk-17-jdk openjdk-17-jre -y
# Verificamos que la instalación de Java sea correcta imprimiendo su versión en
consola java -version
# Creamos el directorio estándar donde se almacenará el SDK de Android de forma
aislada mkdir -p HOME/Android/Sdk
# Configuramos las variables de entorno en el archivo de configuración del
perfil del usuario (.bashrc)
# Esto permite que las herramientas de compilación de Android sean accesibles
desde cualquier terminal echo '
export ANDROID_HOME=HOME/Android/Sdk
' >> HOME/.bashrc echo '
export PATH=PATH:ANDROID_HOME/emulator
' >> HOME/.bashrc echo '
export PATH=PATH:ANDROID_HOME/platform-tools
```



```
' >> HOME/.bashrc
# Recargamos La configuración del archivo .bashrc para aplicar Los cambios de
inmediato sin reiniciarsource HOME/.bashrc
# Comprobamos que La variable de entorno ANDROID_HOME apunte correctamente al
directorio creadoecho ANDROID_HOME
```

Estructura del Android SDK

El SDK de Android no es un bloque único de software, sino un conjunto de herramientas modulares administradas por el **SDK Manager**:

1. **SDK Platforms:** Contiene las librerías específicas de la API de Android correspondientes a una versión del sistema operativo (por ejemplo, API 34 para Android 14). Es obligatorio compilar el código contra una plataforma específica.
2. **SDK Tools:** Contiene herramientas esenciales como adb (Android Debug Bridge), binarios de emulación y el sistema de construcción de software Gradle.

1.5. CONFIGURACIÓN DE DISPOSITIVOS FÍSICOS Y EMULADORES (AVD)

Una vez que el entorno de desarrollo está instalado, el Técnico debe contar con un entorno de pruebas controlado para depurar y certificar el comportamiento de la aplicación móvil.

1. Configuración de Emuladores (AVD - Android Virtual Device)

Un AVD es una instancia de un dispositivo móvil emulada por software en la computadora. Utiliza la tecnología **QEMU** para recrear la arquitectura del procesador y los componentes de hardware (pantalla, acelerómetro, red).

1. **Aceleración por Hardware:** Para que un emulador funcione de manera fluida y no congele la computadora de desarrollo, es crítico activar la virtualización en la BIOS de la computadora (Intel VT-x o AMD-V). En entornos Linux, Android Studio utiliza **KVM (Kernel-based Virtual Machine)** de forma nativa para ejecutar el emulador directamente sobre las instrucciones de la CPU de la computadora, eliminando la sobrecarga de emulación clásica.



2. Configuración de Dispositivos Físicos Real (Método Profesional Obligatorio)

Para proyectos optimizados en el contexto local de Pailón, las pruebas en dispositivos físicos reales son irremplazables, ya que permiten medir el consumo real de batería y el impacto térmico del entorno sobre la aplicación.



Para preparar un teléfono físico para desarrollo, se sigue estrictamente el siguiente protocolo técnico:

1. **Acceso al Menú Oculto:** En el dispositivo móvil, ingresar a *Ajustes* -> *Acerca del teléfono* -> *Información de software*.
2. **Activación de Permisos:** Presionar consecutivamente **7 veces** sobre el campo **Número de compilación**. El dispositivo mostrará un mensaje flotante indicando: "*¡Ahora eres desarrollador!*".
3. **Habilitación de Depuración:** Regresar al menú principal de *Ajustes*, acceder a la nueva sección *Opciones de desarrollador* y activar el interruptor de **Depuración USB**.
4. **Verificación e Intercambio de Claves Criptográficas (ADB):** Al conectar el teléfono a la computadora mediante un cable de datos de alta calidad, el dispositivo solicitará autorizar



la huella digital de la clave RSA de la computadora de desarrollo para permitir el control remoto.

Para verificar y certificar la conexión a nivel profesional desde la consola de comandos, se utiliza el siguiente bloque técnico de instrucciones:

Bash

```
# Iniciamos el servidor de Android Debug Bridge (ADB) y listamos los dispositivos conectados
# Este comando interroga al demonio del sistema para verificar conexiones USB o inalámbricas activasadb devices
# EXPLICACIÓN DEL RETORNO ESPERADO EN CONSOLA:
# Si el dispositivo físico está bien configurado con permisos RSA, la terminal devolverá algo como:
# List of devices attached
# 52003c2d4a51b419    device
## Si el dispositivo aparece como "
unauthorized", significa que se debe aceptar el mensaje de clave RSA en la pantalla del celular.
# Comando avanzado para abrir una terminal interna (shell) dentro del sistema operativo del teléfono físico conectado
# Esto permite examinar los directorios, procesos y recursos internos del dispositivo real desde la computadora del CEAadb shell pm list packages | grep 'example'
# (El comando anterior lista los paquetes instalados en el teléfono que contengan la palabra 'example')
```

1.6. AVANZADO: SINCRONIZACIÓN OFFLINE-FIRST Y EDGE COMPUTING MÓVIL

En la actualidad del desarrollo móvil profesional, la conectividad no se asume como una constante. En municipios con brechas de infraestructura digital, como las áreas periféricas de Pailón (Barrio María Belén, San Antonio o áreas productivas remotas), el diseño técnico debe adoptar la arquitectura **Offline-First**.

Esta arquitectura dicta que la aplicación móvil debe interactuar única y exclusivamente con una base de datos local embebida (como **Room Database** en Android o **Isar/Hive** en entornos híbridos) alojada en el almacenamiento interno del dispositivo. Un componente especializado en segundo plano (como *WorkManager* de Android) monitorea de forma nativa el estado de la red. En el momento exacto en que el transportista ingresa al área de cobertura de antenas en el Barrio



Central Fátima o Barrio 27 de Mayo, el sistema inicia un proceso de sincronización delta (enviando únicamente los datos modificados) hacia servidores en la nube utilizando protocolos eficientes y de bajo consumo de ancho de banda como **gRPC** o **MQTT**, en lugar de peticiones HTTP tradicionales de gran peso en datos.

1.7. AVANZADO: INTELIGENCIA ARTIFICIAL EMBEBIDA (ON-DEVICE AI)

Una de las tendencias disruptivas más importantes en la ingeniería de software móvil moderna es la migración de modelos de Inteligencia Artificial directamente al hardware del dispositivo, eliminando la dependencia de servidores externos y reduciendo a cero el costo en planes de datos para los usuarios locales.

A través de tecnologías como **Google AI Edge** y **TensorFlow Lite**, los Técnicos pueden integrar modelos de clasificación de imágenes, reconocimiento de texto o procesamiento de lenguaje natural optimizados para ejecutarse de forma nativa dentro de los núcleos de hardware específicos del SoC (como las **NPU - Neural Processing Units**). En el caso del proyecto logístico de Pailón, un modelo embebido de TensorFlow Lite podría permitir al transportista escanear de forma instantánea mediante la cámara del teléfono las guías de despacho físicas impresas en papel, extrayendo los datos de texto de forma local, validando las cantidades de los productos sin consumir un solo megabyte de internet del usuario.

1.8. AVANZADO: SEGURIDAD Y ENCLAVES DE HARDWARE AISLADOS

El manejo de transacciones comerciales, datos de inventario y credenciales de usuario exige estándares rigurosos de protección de la información. Las aplicaciones móviles modernas no deben almacenar contraseñas ni datos sensibles en texto plano dentro del almacenamiento regular del dispositivo.

Los sistemas operativos modernos aíslan la generación y resguardo de claves criptográficas utilizando componentes físicos independientes del procesador principal, conocidos como **Trusted Execution Environment (TEE)** en sistemas Android o **Secure Enclave** en dispositivos Apple. A través de la API **Android Keystore System**, un Técnico puede generar claves de cifrado asimétricas (públicas y privadas) protegidas directamente por el hardware del teléfono. El software desarrollado puede requerir de forma obligatoria la validación biométrica local del



usuario (huella dactilar o reconocimiento facial nativo) antes de permitir que el hardware firme digitalmente una orden de entrega o descrypte tokens de acceso confidenciales, garantizando que los datos sigan seguros incluso si el dispositivo móvil es sustraído físicamente en las rutas de transporte.

CIERRE TEÓRICO OBLIGATORIO

❑ *Mitos vs. Realidades*

Mito Técnico Común	Realidad Arquitectónica Demostrada
Las aplicaciones Híbridas son siempre lentas y no sirven para proyectos comerciales serios en hardware real.	Falso. Frameworks modernos como Flutter compilan directamente a código de máquina nativo del procesador. Si están bien optimizados en el manejo de estados de memoria, su rendimiento es indistinguible de una aplicación nativa para el usuario común.
Para que una aplicación móvil funcione sin internet, basta con guardar los archivos de texto en la memoria SD del teléfono.	Falso. La lectura en almacenamiento externo sin estructurar genera fallos de consistencia, riesgos de seguridad severos y lentitud extrema. El desarrollo moderno exige el uso de bases de datos relacionales locales con soporte ACID gestionadas mediante ORMs profesionales.

❑ *Glosario de Términos*

1. **SoC (System on Chip):** Circuito integrado de dimensiones milimétricas que aloja en una única pieza de silicio todos los componentes vitales de procesamiento (CPU, GPU, módems de red, memoria y controladores) de un dispositivo móvil.
2. **SDK (Software Development Kit):** Conjunto especializado de herramientas de software, APIs de desarrollo, compiladores, binarios de depuración y documentación provistos por un fabricante para permitir la construcción de aplicaciones sobre su plataforma.
3. **AVD (Android Virtual Device):** Configuración de software que define las características técnicas de un teléfono o tableta simulados dentro de la computadora de desarrollo para ejecutar pruebas de código de alta fidelidad.



4. **PWA (Progressive Web App):** Aplicación web construida con estándares modernos que, mediante el uso de Service Workers, adquiere capacidades de software instalado, como funcionamiento sin conexión e instalación directa en el sistema operativo del teléfono.
5. **ADB (Android Debug Bridge):** Herramienta de línea de comandos sumamente versátil que actúa como un puente de comunicación bidireccional entre la computadora de desarrollo y el dispositivo Android físico o emulado, permitiendo tareas de control, instalación de software y depuración profunda.

□ **Ponte a Prueba**

1. **¿Qué componente de software de las PWAs actúa como un intermediario de red y permite que la aplicación funcione completamente sin conexión a internet?**

1. A) El Android Virtual Device (AVD).
2. B) El Service Worker.
3. C) El recolector de basura (Garbage Collector).
4. *Respuesta Correcta: B.* Justificación: Los Service Workers operan en segundo plano de manera independiente al navegador y son los encargados de interceptar las peticiones de red y servir los recursos almacenados en la caché local cuando no hay conexión a internet disponible.

2. **¿Por qué un procesador móvil con arquitectura ARM big.LITTLE distribuye las tareas en diferentes tipos de núcleos?**

1. A) Para aumentar el peso gráfico de las interfaces de usuario de forma simultánea.
2. B) Para optimizar la disipación pasiva de calor y equilibrar el rendimiento extremo con la conservación drástica de la batería.
3. C) Para permitir la instalación concurrente de múltiples sistemas operativos de código abierto.
4. *Respuesta Correcta: B.* Justificación: Esta arquitectura asigna las tareas cotidianas de bajo consumo a los núcleos eficientes y reserva los núcleos de alta frecuencia únicamente para



operaciones demandantes, evitando el sobrecalentamiento del dispositivo móvil en entornos cálidos.

3. **¿Cuál es el comando correcto de la consola ADB que permite verificar formalmente si un teléfono celular físico conectado por USB ha establecido comunicación exitosa en modo desarrollador con la computadora?**

1. A) adb start-server
2. B) adb shell pm list
3. C) adb devices
4. *Respuesta Correcta: C.* Justificación: El comando adb devices interroga de manera directa al servidor de depuración e imprime en pantalla el identificador único de cada dispositivo conectado junto con su estado actual (device, unauthorized, u offline).



VALORACIÓN



REFLEXIÓN DE IMPACTO TÉCNICO Y SOCIAL EN LA REGIÓN

Como Técnicos en Sistemas Informáticos formados en el contexto del **CEA Herman Ortiz Camargo**, la adquisición de habilidades en programación y desarrollo móvil no debe desligarse de una profunda responsabilidad ética y socioambiental hacia nuestra comunidad en Pailón.

El diseño y optimización de software tiene un impacto directo en la problemática global de los **desechos electrónicos (e-waste)**. Cuando un desarrollador construye software pesado, ineficiente y mal optimizado, obliga de manera indirecta a que los usuarios descarten sus teléfonos celulares antiguos de gamas de entrada (los cuales terminan acumulándose en vertederos locales e impactando los suelos de la provincia Chiquitos con metales pesados como litio, plomo y cadmio), debido a que la aplicación "ya no entra" o "congela el teléfono". Crear software optimizado, liviano y retrocompatible es un acto de **sostenibilidad ecológica**, extendiendo el ciclo de vida útil del hardware existente de los ciudadanos de Pailón.



Asimismo, la inclusión tecnológica representa un factor social crítico. Al diseñar aplicaciones móviles para el comercio o el transporte local que sean capaces de funcionar eficientemente bajo las condiciones de conectividad intermitente de las áreas rurales y periféricas, el Técnico está rompiendo activamente las brechas digitales, permitiendo que pequeños productores locales compitan en mercados formales en igualdad de condiciones, dinamizando la economía de la región bajo principios éticos de inclusión y equidad social.



PRODUCCIÓN



LABORATORIO PROFESIONAL: CONFIGURACIÓN DEL ENTORNO DE COMANDOS Y DIAGNÓSTICO DE DISPOSITIVOS DE DESARROLLO MÓVIL

Objetivo General

Configurar e inspeccionar mediante consola el entorno técnico de desarrollo y certificar el enlace bidireccional entre la estación de trabajo y un dispositivo físico real mediante el puente ADB, simulando las condiciones de desarrollo del laboratorio de computación del CEA Herman Ortiz Camargo.

Equipamiento Requerido

1. Computadora de escritorio del laboratorio equipada con sistema operativo basado en Linux o Windows.
2. Cable de datos USB de alta calidad (preferentemente el cable original del dispositivo).
3. Dispositivo móvil Android físico provisto por el estudiante.

Guía Paso a Paso Ejecutable

Paso 1: Inicialización del Entorno de Trabajo y Variables Globales

Abra una terminal de comandos en su computadora y ejecute el siguiente bloque de comandos técnicos para automatizar la creación de una herramienta de script que configure y valide de manera instantánea el directorio raíz del SDK de desarrollo.



Bash

```
# Escribimos un script de automatización local llamado '
validar_entorno.sh'
cat << '
EOF
' > validar_entorno.sh
#!/bin/bash
# Script de diagnóstico técnico de dependencias de desarrollo móvilecho
"=====
echo "      INICIANDO DIAGNÓSTICO DE ENTORNO EN EL CEA      "
echo "=====
# Comprobación estricta de la instalación de Java en el sistemaif type -p java >
/dev/null;
then     echo "[OK] Java detectado correctamente en la ruta del sistema."     java
-version 2>&1 | head -n 1else     echo "[ERROR] Java no está instalado. Por favor
ejecute: sudo apt install openjdk-17-jdk"
fi
# Validación del estado de la variable de entorno ANDROID_HOMEif [ -z
"ANDROID_HOME" ];
then     echo "[ALERTA] La variable ANDROID_HOME está vacía temporalmente en esta
sesión."     # Asignamos la ruta estándar por defecto para la sesión actual de la
terminal     export ANDROID_HOME=HOME/Android/Sdk     echo "-> Ajustando
temporalmente la ruta a: ANDROID_HOME"
else     echo "[OK] Variable ANDROID_HOME configurada correctamente en:
ANDROID_HOME"
fi
echo "=====
EOF
# Otorgamos permisos explícitos de ejecución al script técnico creado en el
directorio actualchmod +x validar_entorno.sh
# Ejecutamos el script para verificar el estado de la estación de trabajo en
Pailón./validar_entorno.sh
```

Paso 2: Vinculación Física y Autorización de Seguridad en Hardware Real

1. Tome el teléfono celular físico, ingrese a la configuración del sistema operativo y proceda a activar el menú de desarrollo realizando los 7 clics correspondientes sobre el número de compilación tal como se detalló en el sustento teórico (Sección 1.5).
2. Active de forma estricta la **Depuración USB**.
3. Conecte el cable USB a la computadora de desarrollo del CEA. Al encenderse la pantalla del teléfono, aparecerá una alerta de seguridad con el mensaje: "¿Permitir depuración USB?"



La huella digital de la clave RSA de la computadora es...". Marque la casilla de verificación "Permitir siempre desde esta computadora" y presione el botón de **Aceptar**.

Paso 3: Auditoría Técnica y Extracción de Datos de Rendimiento mediante ADB Consola

Con el dispositivo enlazado, ejecute los siguientes comandos especializados en la terminal para diagnosticar las capacidades reales del hardware del teléfono en el que desplegará el software comercial:

Bash

```
# Detenemos de forma segura cualquier instancia colgada del servidor ADB para
limpiar el canal de datosadb kill-server
# Levantamos el servidor de depuración de manera limpia e inspeccionamos los
dispositivos conectadosadb devices
# Extraemos la información específica de la arquitectura del procesador del
teléfono físico
# Esto le indica al Técnico si el celular es de 64 bits (arm64-v8a) o
arquitecturas más antiguascomo "
Arquitectura de CPU del Dispositivo Móvil:"
adb shell getprop ro.product.cpu.abi
# Inspeccionamos el estado de carga y salud térmica de la batería del
dispositivo real conectado
# Es vital monitorear estos valores para evitar fallos térmicos durante pruebas
extremas en Pailóncho "
Estado Físico del Subsistema de Energía y Batería:"
adb shell dumpsys battery | grep -E "
status|health|present|level|temperature"
# NOTA TÉCNICA DE INTERPRETACIÓN:
# El campo '
temperature
' se devuelve en décimas de grado Celsius. # Por ejemplo, un valor de '345
' representa exactamente 34.5°C en el núcleo de la batería.
```

Criterio de Evaluación de Entrega del Reto

El estudiante habrá completado con éxito este laboratorio práctico cuando presente en la pantalla de su terminal el listado explícito de adb devices con el estado en modo device confirmado (sin leyendas de error de autorización), acompañado del reporte impreso de la arquitectura del procesador de su teléfono móvil y la temperatura exacta del dispositivo en el momento de la prueba.



RECURSOS ADICIONALES

1. **Android Developer Documentation (Guía Oficial de Google):** El portal maestro indispensable para todo Técnico que busque profundizar de manera formal en el ciclo de vida de los componentes de Android, el SDK de Gradle y las buenas prácticas de diseño de interfaces. <https://developer.android.com>
2. **Flutter Documentation Hub:** La documentación oficial y estructurada para el diseño híbrido de alto rendimiento multiplataforma, que incluye guías prácticas de instalación y optimización de código fuente. <https://docs.flutter.dev>
3. **MDN Web Docs - Aplicaciones Web Progresivas (PWA):** Repositorio académico completo mantenido por Mozilla, ideal para dominar los fundamentos técnicos, la escritura avanzada de Service Workers y los manifiestos de instalación web móvil. https://developer.mozilla.org/es/docs/Web/Progressive_web_apps